# Performance evaluation of cloud systems and services

Cloud Computing Course

Emiliano Casalicchio

SAPIENZA
UNIVERSITÀ DI ROMA

# Agenda

- Performance evaluation
  - General concept
  - Cloud computing performance evaluation

- How to test scaling in / out
- How to select appropriate scaling metrics
- How to set appropriate threshold values

# Computer systems performance evaluation [1]

- Assesses how efficiently a computer utilizes its resources (CPU, memory, storage) and achieves desired outputs
- Helps identify areas for improvement, optimize resource utilization, and ensure the system meets performance requirements
- The four techniques for computer systems performance evaluation include
  - analytical modeling,
  - Petri-net modeling,
  - Simulation modeling,
  - **Empirical or testbed analysis**

[1] https://doi.org/10.1016/B978-155558260-9/50003-5

# Performance evaluation of cloud systems and services

- Assesses how effectively cloud resources are functioning, helping organizations identify bottlenecks and improve overall system performance
  - Empirical evaluation involves monitoring various metrics, conducting performance testing, and analyzing results to optimize cloud environments

**Key areas of focus**

availability
response times
scalability
resource utilization

**Performance evaluation benefits**

Optimizing Resource Utilization
Ensuring Business Continuity
Improving Scalability and Flexibility
Meeting Service Level Agreements (SLAs)

# Key Metrics to Monitor

- Availability
  - The percentage of time a service is operational and accessible to users.
- Response Time
  - The time it takes for a system to respond to a request or user action. Latency is a key component of response time.
- Throughput
  - The amount of data or work a system can process within a given timeframe.
- Resource Utilization
  - Measures like CPU utilization, memory usage, and disk I/O, indicating how efficiently resources are being used.
- Error Rate
  - The frequency of errors or failures within the system.
- Scalability
  - The ability of the system to handle increasing workloads or demands without compromising performance.

# Performance Testing Techniques

- **Load Testing**
  - Simulates typical workloads to assess how the system performs under normal operating conditions
- Stress Testing
  - Exposes the system to extreme loads to identify weaknesses and capacity limits
- Endurance Testing
  - Tests the system's ability to maintain performance over extended periods
- Spike Testing
  - Simulates sudden bursts of activity to evaluate the system's ability to handle unpredictable load spikes.

# Performance evaluation of cloud services

- A general procedure for measurement-based service performance evaluation on a Cloud testbed is comprised of the following steps [2]
  1. To specify the purpose and scope of the evaluation
  2. To Identify the features/aspects of the Cloud services that are to be evaluated
  3. To determine the performance metrics that will be analyzed
  4. To select appropriate tool for evaluating performance
  5. To design performance evaluation experiments
  6. To setup an experimental environment
  7. To run performance evaluation experiments

[2] https://doi.org/10.1016/j.dcan.2016.12.002

**Two categories of performance measurements [1]**

- System-oriented measures (or server side)
  - Throughput, Utilization, Scalability, Availability
- User-oriented measures (or client side)
  - Response time, Availability, Error rate

# To specify the purpose and scope of the evaluation

- The purpose and scope is very use case specific. Some questions can help in identifying them are the following
- What is the scope of the analysis?
  - To identify bottleneck?
  - To verify SLA are meet?
  - To optimize resource utilization?
  - To ensuring business continuity / high availability?
  - To verify and improve scalability?
  - …
- Am I interested in the performance from a user perspective, or the service provider perspective, or both?

# Project cases 1

*To implement, by means of AWS Lambda functions and AWS CloudWatch, the adaptive step size algorithm presented in …. and to use the step size dynamically determined to scale EC2 instances under continuous and changing workload conditions*

The purpose and scope of the performance evaluation is to assess if and in which measure EC2 instance scale with the implemented algorithm

The perspective is from the point of view of the service provider

# Project case 2, 3, 5

*2) To deploy an image processing application on AWS lambda and to evaluate its performance and scalability under different workload conditions.*

*3) To deploy an AI based application at your choice on AWS lambda and to evaluate its performance and scalability …*

*5) To deploy a scalable and high-available multitier web application that foreseen a data layer (a Database or an object store area) …*

The purpose and scope of the study is to collect performance measures that show

- The application scale when the workload increase
- How many resources are needed to guarantee a certain level of service when the workload increase
  - Resources could be lambda functions, EC2 instances, DB instances,
- If there are hot spot functions (potential bottlenecks)
- What is the throughtput and the response time
- If the deployment is high available

The perspective is both from the client side and server side

# Project case 4

*To setup a Kubernetes cluster on AWS EC2 instances (do not use EKS), configure the pod autoscaling module and show that a simple dockerized web application scales out and in under different workload conditions*

The purpose and scope of the study is to collect performance measures that show

- The application scale when the workload increase and decrease

# To Identify the features/aspects of the Cloud services that are to be evaluated

- The deployment of a complex cloud service include many components and features (functionalities)
- To make an example a scalable and high available web application can include
  - The web application itself (the application logic)
  - A front end web server
  - A data store layer (database / file server)
  - An authentication service
  - A content delivery service
- A performance study can focus on all, few or one of the components
- Determining the features allows to determine what is the workload to be considered, what specific performance metric to collect and how

# Example

- Project case 2 and 3
  - The features to be considered are the Lambda function (one or more) that implement the image processing phase
  - Authetication service, data store layer, front-end web server, accuracy/precision of the model could be kept out of the analysis
    - Assuming they do not represent a botlleneck for the system
- Project case 5
  - Under analysis should be the web application logic and front-end layer assuming the database/data store layer is scalable
    - This assumption is mainly done because analyzing the scalability of database/datastore require to generate very intesive workload

# To determine the performance metrics that will be analyzed

- User oriented
  - Availability
  - Response Time
  - Error Rate
- System oriented
  - Throughput
  - Resource Utilization
  - Scalability
  - Availability

The measurement of these metrics depends on the specific system component considered.
Let us consider the throughput:

- DB – transations/sec
- Disk – bytes_read/sec, blocks_read/sec, bytes_write/sec, blocks_write/sec
- Generic application – requests/sec

*AWS cloud watch define metrics for each service*

# Metrics available in AWS

- The list of AWS service that publish cloud metrics is available here https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/aws-services-cloudwatch-metrics.html

# Example of EC2 metrics

- CPUUtilization
- DiskReadOps
- DiskWriteOps
- DiskReadBytes
- DiskWriteBytes
- NetworkIn
- NetworkOut

All Ops/byte from/to attached volumes and NIC

# Example of Autoscaling group metrics

- GroupMinSize
- GroupMaxSize
- GroupInServiceInstances
- GroupPendingInstances
- GroupStandbyInstances
- GroupTerminatingInstances

# Examples Lambda metrics

- Divided in five topics
  - Invocation metrics
    - Invocations, Errors, DestinationDeliveryFailures, Throttles
    - …
  - Performance metrics
    - Duration
    - …
  - Concurrency metrics
    - ConcurrentExecutions
    - …
  - Asynchronous invocation metrics
    - AsyncEventsReceived
    - AsyncEventAge
    - AsyncEventsDropped
  - Event source mapping metrics

# How to use this metrics for System-oriented evaluation (examples)

- Scalability
  - GroupInServiceInstances, ConcurrentExecutions as a function of time and workload fluctuations
- Throughput
  - NetworkOut/second
  - Disk*Ops/second
    - This is the total write/read file system throughput, if you want to focus on the throughput for a specific volume, you need to select the metric for the specific volume (e.g. EBS)
  - (Invocations – Errors – Throttles)/second
- Availability
  - 1 – Throttles/total_num_of_requests
  - 1 – StatusCheckFailed/number_of_observations
    - (is evaluated every minute by default)

# Metrics for User-oriented evaluation

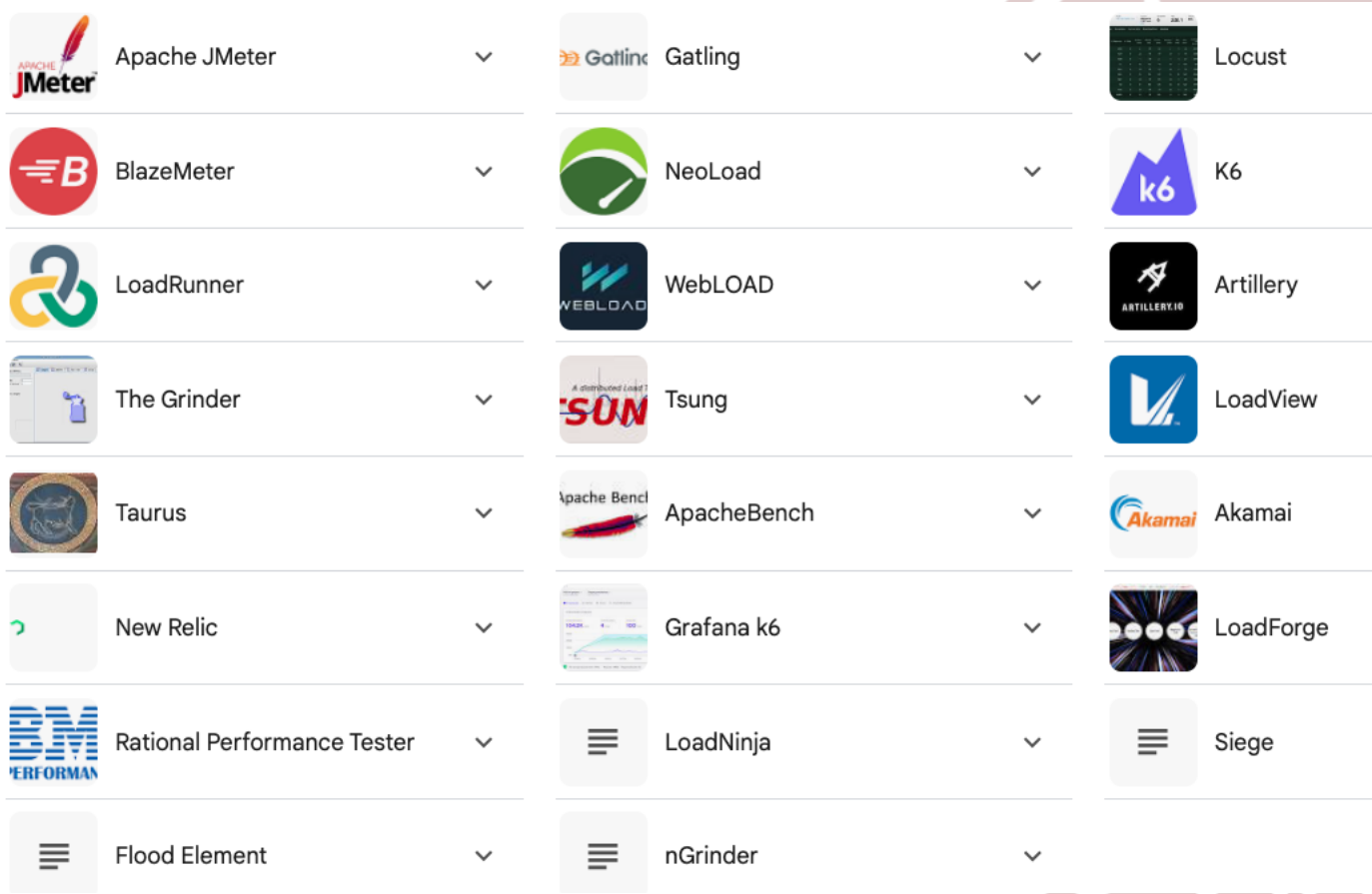
- Must be collected on the client side, typically measured by the tool used to generate the workload
- With a load generator you can collect metrics like
  - Response time / latency
    - Time the answer is received – time the request is sent
  - Successful requests
  - Throughput
    - Number of requests served x time unit (e.g. minute)
  - Traffic (byte or reqs in/out)
  - Availability
- All the above metrics
  - Are measured by the client (who generate the requests) point of view
  - Are influenced by everything between the client and the server (i.e. the internet)

# To select appropriate tool for evaluating performance

- The choice depend on the *features/aspects of the Cloud services that are to be evaluated*
- The features identified allow determine, for example, if requests should be generated for
  - API
  - Web server
  - Database server
  - Disk
- Moreover, the feature and scope allow to identify the workload to be generated
  - Type of requests
  - Intensity (how many requests per second)
  - Duration of the experiment

# Tools for evaluating Web application performance (All Projects)

- Many tools are available
  - Jmeter
  - Locust
  - Artillery
  - ApacheBench
- Typically allow to define, in a configuration file, the duration of the test, type and frequency of requests, and collect metrics such as throughput, response time, errors
  - ApacheBench is a command line tool that allow to send only one type of request



# What to consider for selecting the performance evaluation tool (examples)

- An image processing application deployed with Lambda
  - Requests can be directly sent to the API gateway, no need to go through a web page that allows to upload manually images
  - Requests should be for different type of images (e.g. different size)
- A web application
  - Requests should go through the web front-end, the web application logic server, and eventually the data store layer
  - Requests should sollicit all the functionalities offered by the web application, included the access to the data store layer (eventually)
    - A pool of different type of requests should be carefully selected

The selected tool should allow to meet the above requeriments

# To design performance evaluation experiments

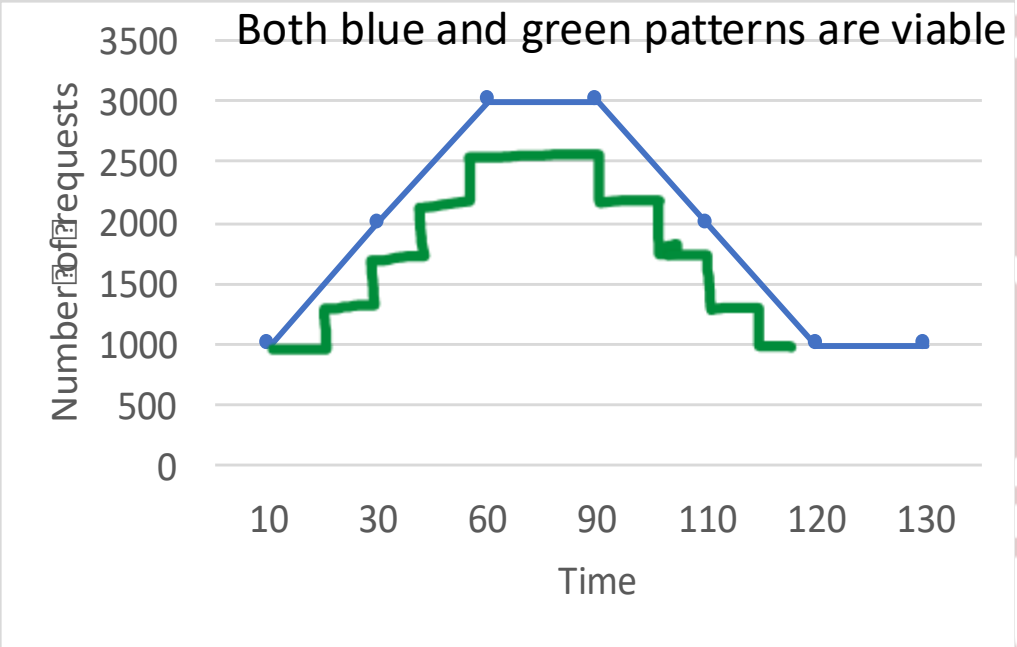
- Driven by the outcome of steps 1 – 3
  - purpose and scope, features/aspects, performance metrics
- Experiments are characterized by
  - The workload generated
    - Type of requests (CPU intensive, Disk Intensive, Memory Intensive, Network Intensive)
    - Intensity
  - The duration
    - For how many time the system is subject to the desired workload
  - The number of time a single run (experiment) is executed
    - You can not rely on a single outcome

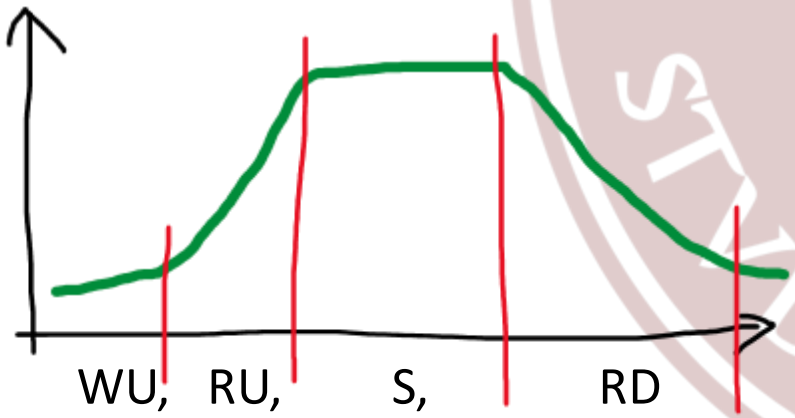
# Example: an image processing application

- The workload generated
  - Type of requests should solicit the application in different way, e.g.
    - Images size range from few pixels to millions of pixels, or
    - Images complexity range from few objects to hundreds of objects
  - Intensity
    - Theory: from few to hundreds or thousands requests per seconds (e.g. 10 to 500 req/sec)
    - **Practice: AWS free tier limit the throughput and AWS learning lab have many limitations; a too intensive workload results in account banning --> limit the workload intensity e.g. 10 – 50/100**
- The duration
  - You need to observe the system and collect a considerable number of datapoints for the selected metrics
  - If datapoints are generated every minute, run the experiment for 10-30 minutes
- The number of time a single run (experiment) is executed
  - A single outcome is not enough to draw conclusions
  - Theory: at least 20
  - **Practice: you have a limited budget in time and AWS credits, hence running 3 – 5 experiments should allow you to draw appropriate conclusions on the outcome of the expriments**

# Workload shape

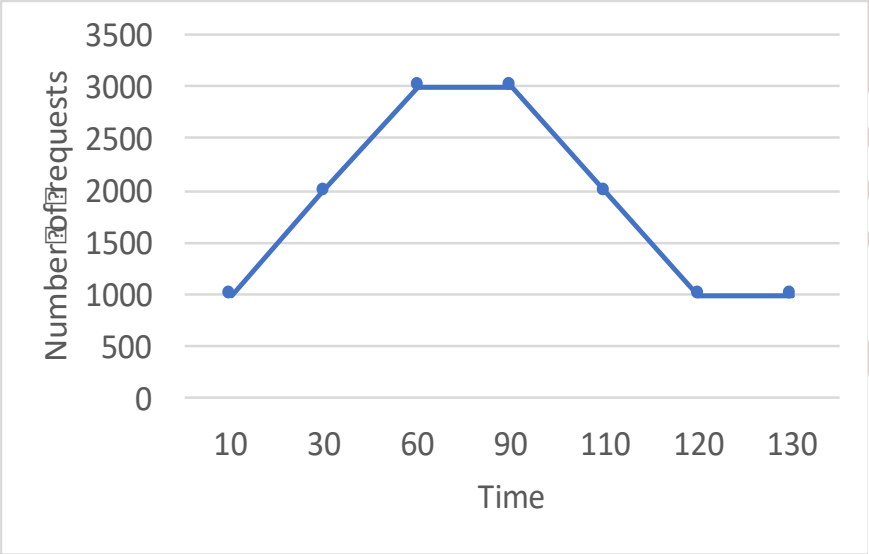
- Continuously increase/decrease the load
- The test is composed of the following phases
  - Worm-up (WU) period
  - Rump-up (RU) period, load increase until a certain amount
  - Steady (S) period, load is stable at the max value
  - Rump-down (RD) (to test scale down)
- The increase of the load should be continuous.
  - There are stress tools that allow to automatically increase the load upon a certain amount
  - Other benchmark require you run it many times with increased load values
    - Tip: Do not let cooldown the system between two runs



# Workload specifications depends on the tool

- The following shell script increases & decreases the number of requests executing 5 times the ab command with varying parameters
  - `for req in 1000 2000 3000 2000 1000; do ab -n $req -c 10 http://ip-10-251-50-12.ec2.internal/index.html; done`
  - It could be used to stress a very simple web server
  - Prons: simple
  - Cons: you request always the same page
- More sophisticated tools like Jmeter allow to specify rump-up, steady, rump-down periods and test many different applications/server/protocol types, e.g.
  - Web - HTTP, HTTPS (Java, NodeJS, PHP, ASP.NET, …)
  - REST Webservices
  - Prons: full control
  - Cons: could be hard to configure
- Alternatively web stress test tools like Artillery or Loadster
  - Prons: easy to configure
  - Cons: not full control, risk to generate too much load an get banned by AWS



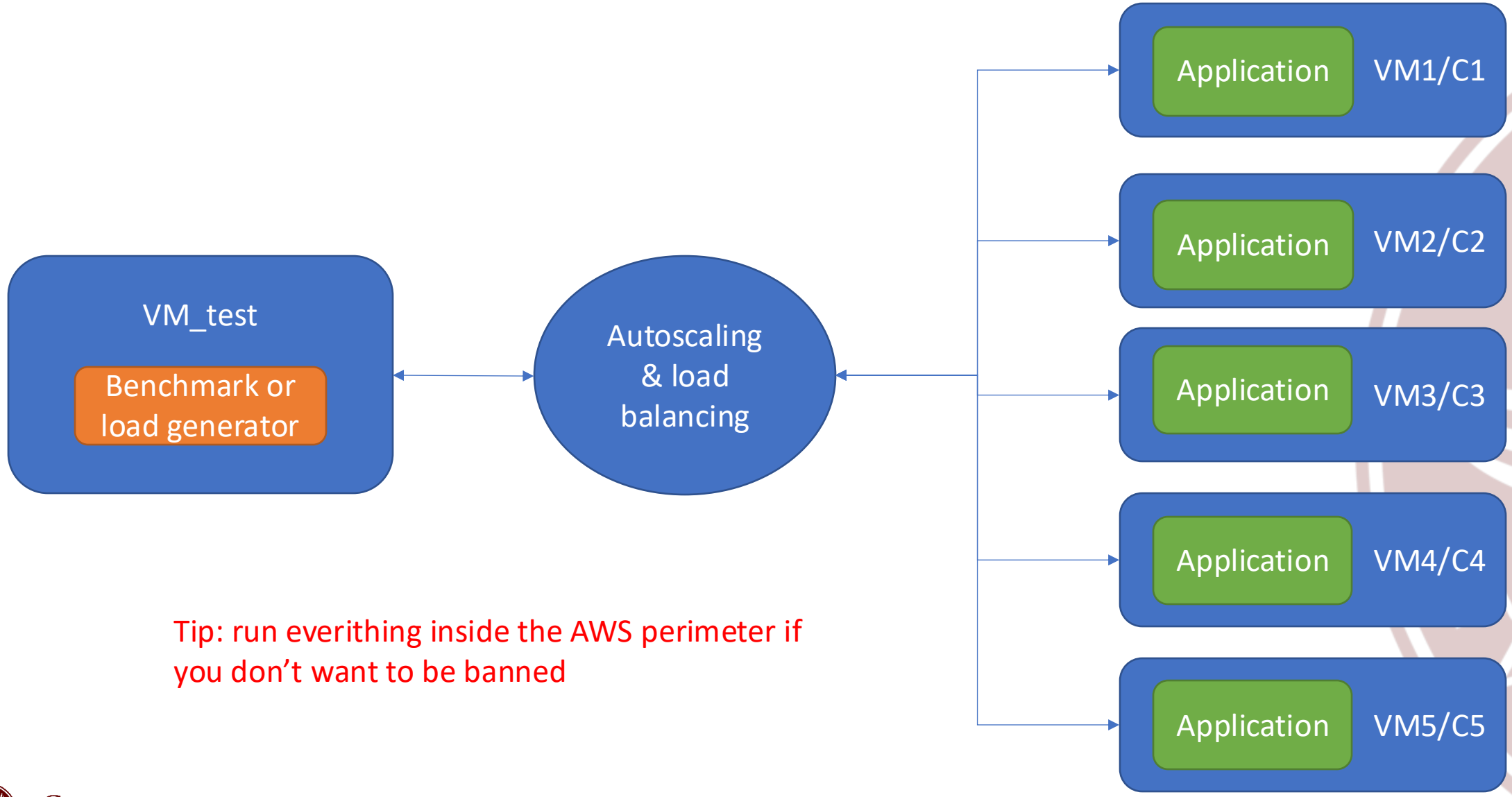
# Challenge: to reproduce the user behavior

- Solution 1
  - Select a set of representative requests
  - Assign a probability to each request
  - Partition the load on the base of the probability selected, e.g.:
    - P(Req1)=0.3 P(Req2)=0.5 P(Req3)=0.2
    - If you generate 100 requests/sec, they will be distributed as follow 30 Req1/sec 50 Req2/sec and 20 Req3/sec
- Solution 2
  - Use a tool like Selenium WebDriver which allow to reproduce user behavior

# To setup an experimental environment

- Ensure the application/deployment is bug free
- Setup a node to run the workload generation tool that is inside the AWS perimeter
  - Needed to avoid bunning
  - *Does not allow to collect accurate user-oriented measurements. In theory the workload generation tool should be on a remote node or even better distributed among many remote nodes (outside the cloud perimeter)*
- Configure AWS CloudWatch to collect all the planned metrics
- Prepare the configuration file for your workload generation tool
  - TIP: Do not generate a too intensive workload

# Typical web app testing configuration




Tip: run everithing inside the AWS perimeter if you don't want to be banned

# To run performance evaluation experiments

- For each experiment collect the datapoints for the selected metrics
  - Dowload the collected datapoints using the consolle feature or a CLI script
- Average the value of the collected datapoints over all the experiments
- Plot the everage value
- Example
  - You run an experiment for 10 minutes and you collect metric datapoints each minute, then you have 10 datapoints
  - You ran 5 experiments

| Run | Dp1 | DP2 | … | DP10 |
|---|---|---|---|---|
| 1 | V1.1 | V1.2 | | V1.10 |
| 2 | | | | |
| 3 | | | Vi.j | |
| 4 | | | | |
| 5 | V5.1 | | | V5.10 |
| AVG | *V1* | *V2* | *Vj* | *V10* |

# For threshold-autoscaling: How to set appropriate threshold values

- Is typically a heuristic process
  - The rule of thumb (practitioner experience) - 66% of maximum capacity
  - Results of simulation studies
  - Adaptive thresholds
- In our case
  - For CPU utilization
    - 66%-70% is the safe level of utilization for a system
    - Hence you can setup threshold around this value
  - For Network traffic (https://cloudonaut.io/ec2-network-performance-cheat-sheet/)
    - t2.micro, 0.06Gbit/sec=7500Kbyte/sec - 5.06Gbit/sec=0.63Gbyte/sec
    - Hence you can setup a threshold around 0.63Gbyte/sec*0.66=0.41Gbyte/sec

# How to set appropriate threshold values to **show** that **your system** scale in/out: Examples

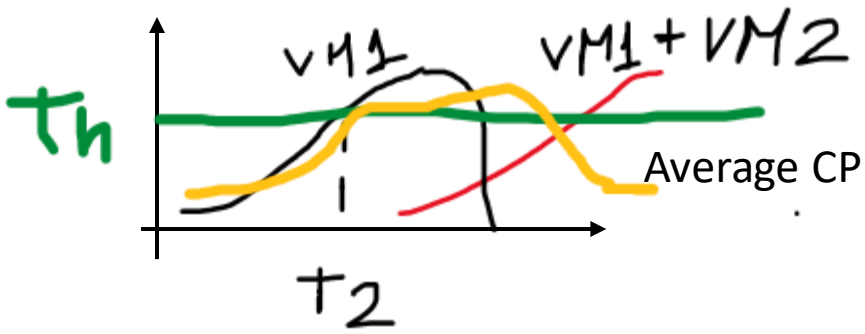
- If under the load that you generate, your application demand for a maximum CPU utilization = 50% you can setup a threshold at 35%
- If your application generate maximum 3000 req/sec you can setup a threshold at 2000 req/sec
- … and so on
  - I have always applied the 66% rule on the maximum value

# How to test availability

- Generate random failures by
  - killing services or
  - stopping VMs/containers

# A common mistake: using CPU stress tools to test scalability

- T1: You run CPU stress tool in a VM instance (VM1)
- T2: the CPU load reaches the threshold on CPU utilization and you scale
- T3: a new instance (VM2) starts and the CPU stress process run automatically
  - In the meantime probably the CPU stress process on VM1 is terminated




Average CPU load, when you have 2 VMs running, but the full load is only on a machine

- Hence
  - the CPU stress tool should run on each instance from the start of the instance to the end of the experiment
- Limitation
  - You can reproduce only the WU e RU phases

# Other common mistakes

- An application that allow to post stories (text or images)
  - **You cannot evaluate** the system performance only with get requests (i.e. downloading repeatedly the front-end web page)
  - You must generate a mix of requests that upload text and images of different size
- An application that have a light fronted but make many read and write request to a database
  - Evaluating the scalability/performance of the fronted is meaningless
  - You must generate a mix of DB R/W requests